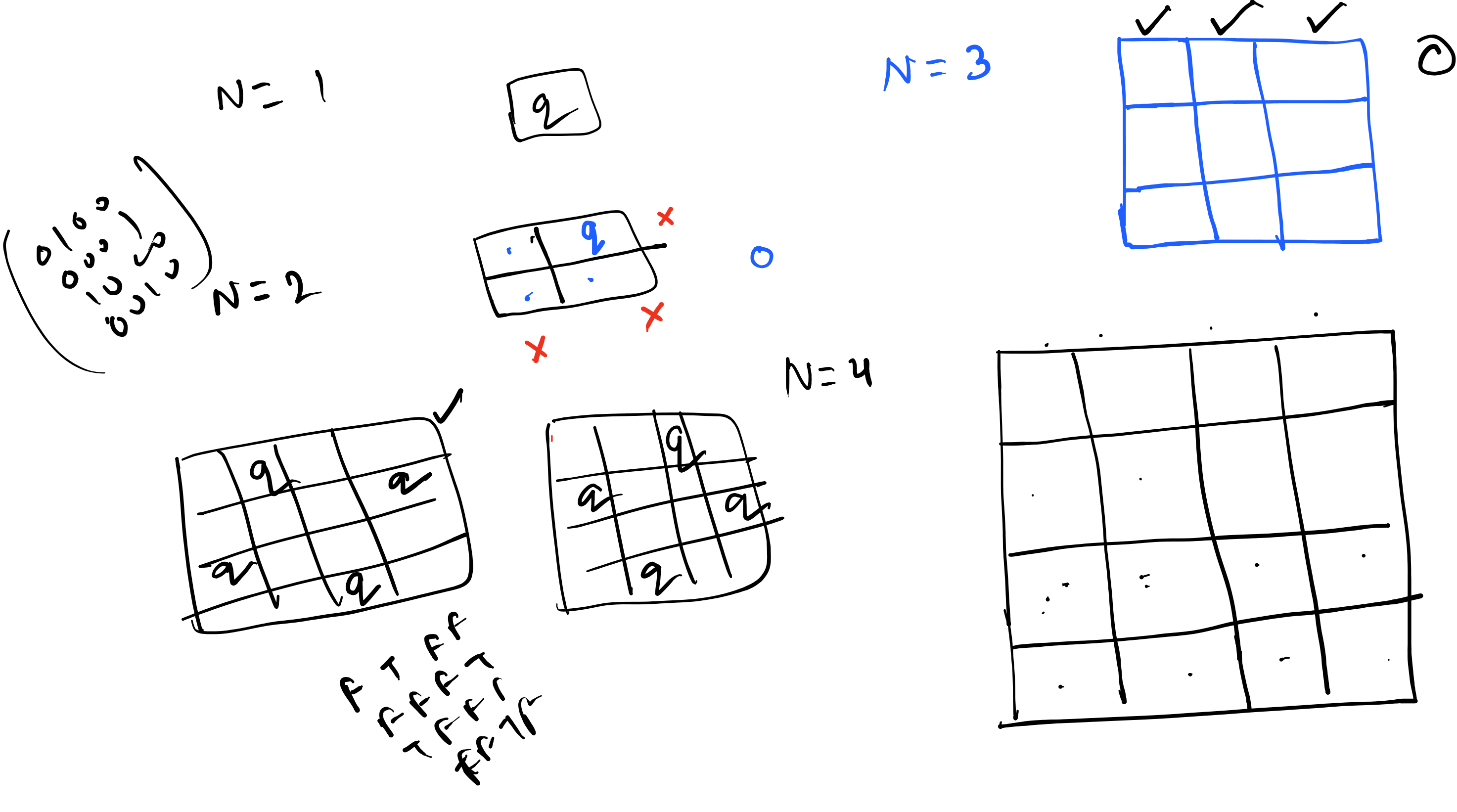
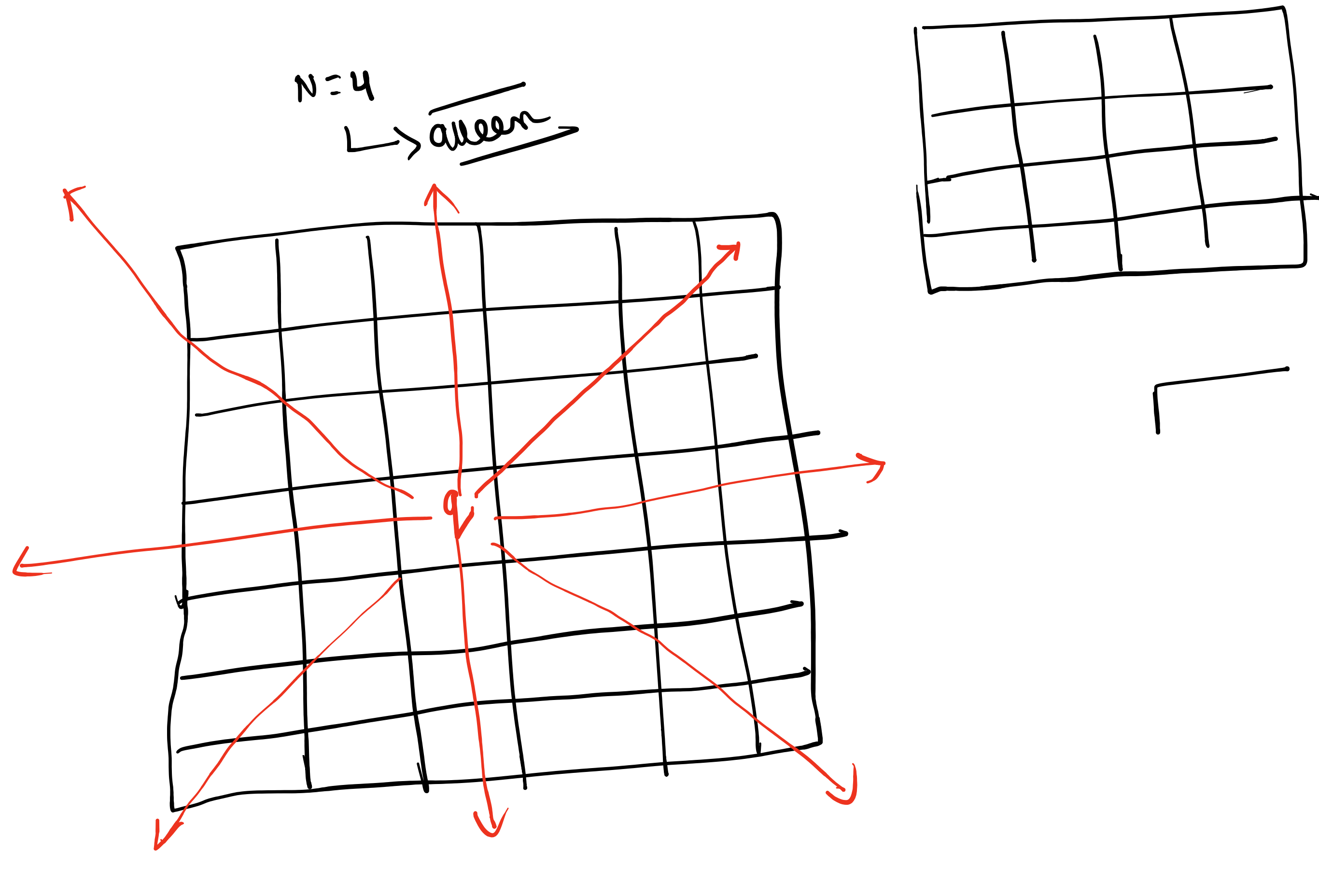
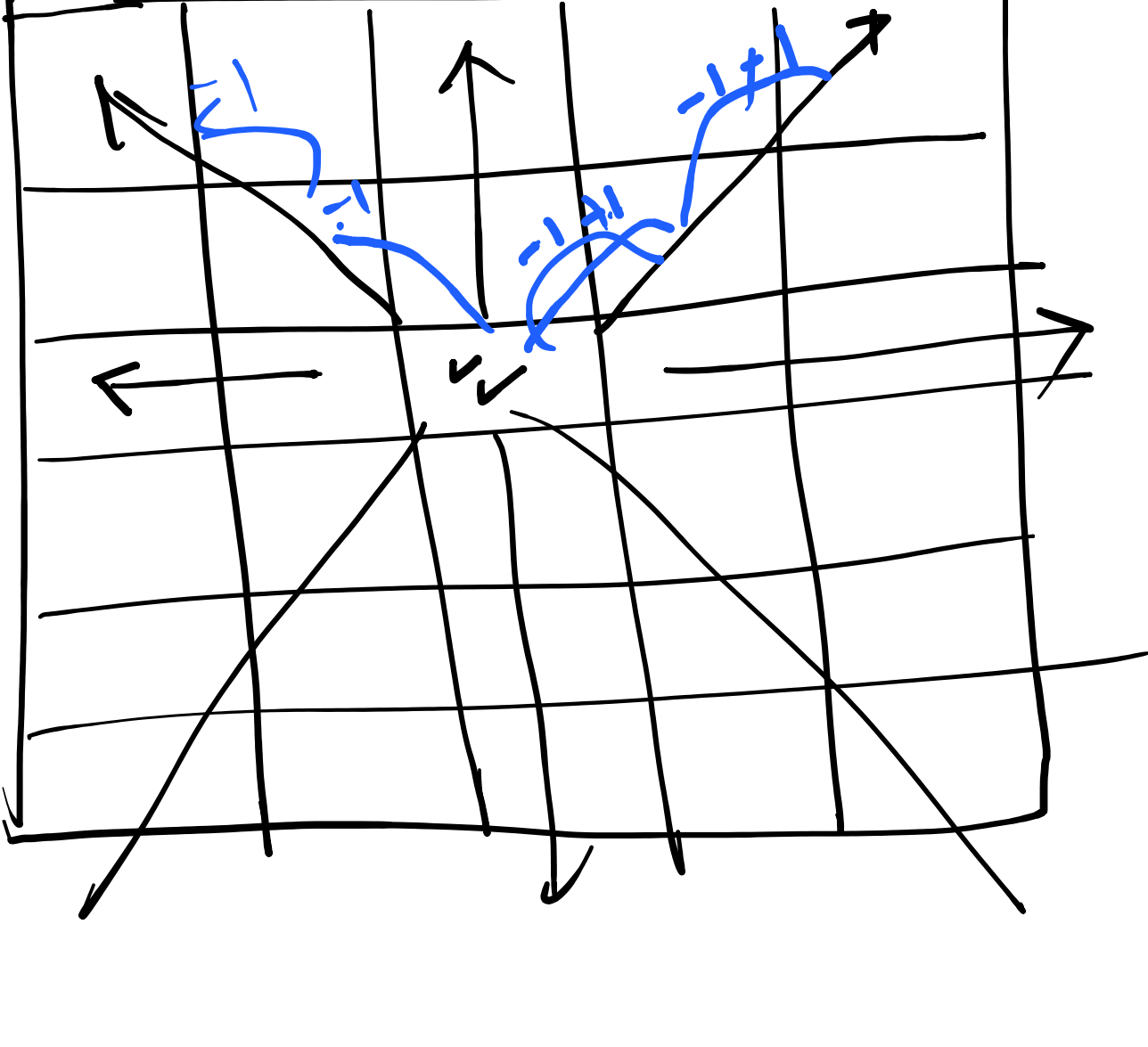
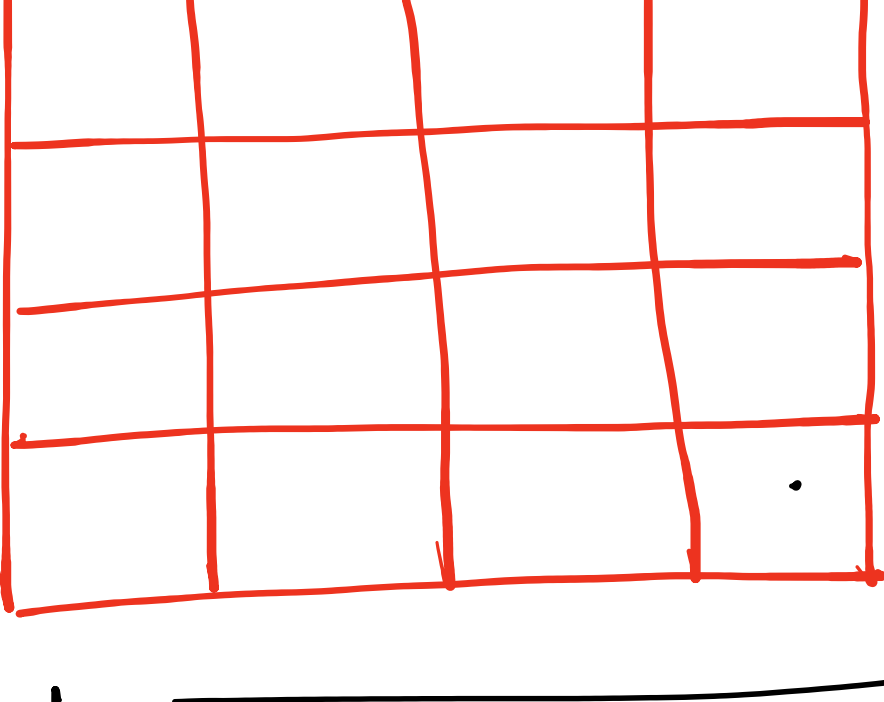




```
public static void Print(boolean[][] board, int tq, int row) {
    if (tq == 0) {
        Display(board);
        return;
    }
    for (int col = 0; col < board.length; col++) {
        if (isitpossible(board, row, col)) {
            board[row][col] = true;
            Print(board, tq - 1, row + 1);
            board[row][col] = false;
        }
    }
}
```

P T F F
F F F T
T F F F
F F T F

(3,1)
2,2
1,3
0,4



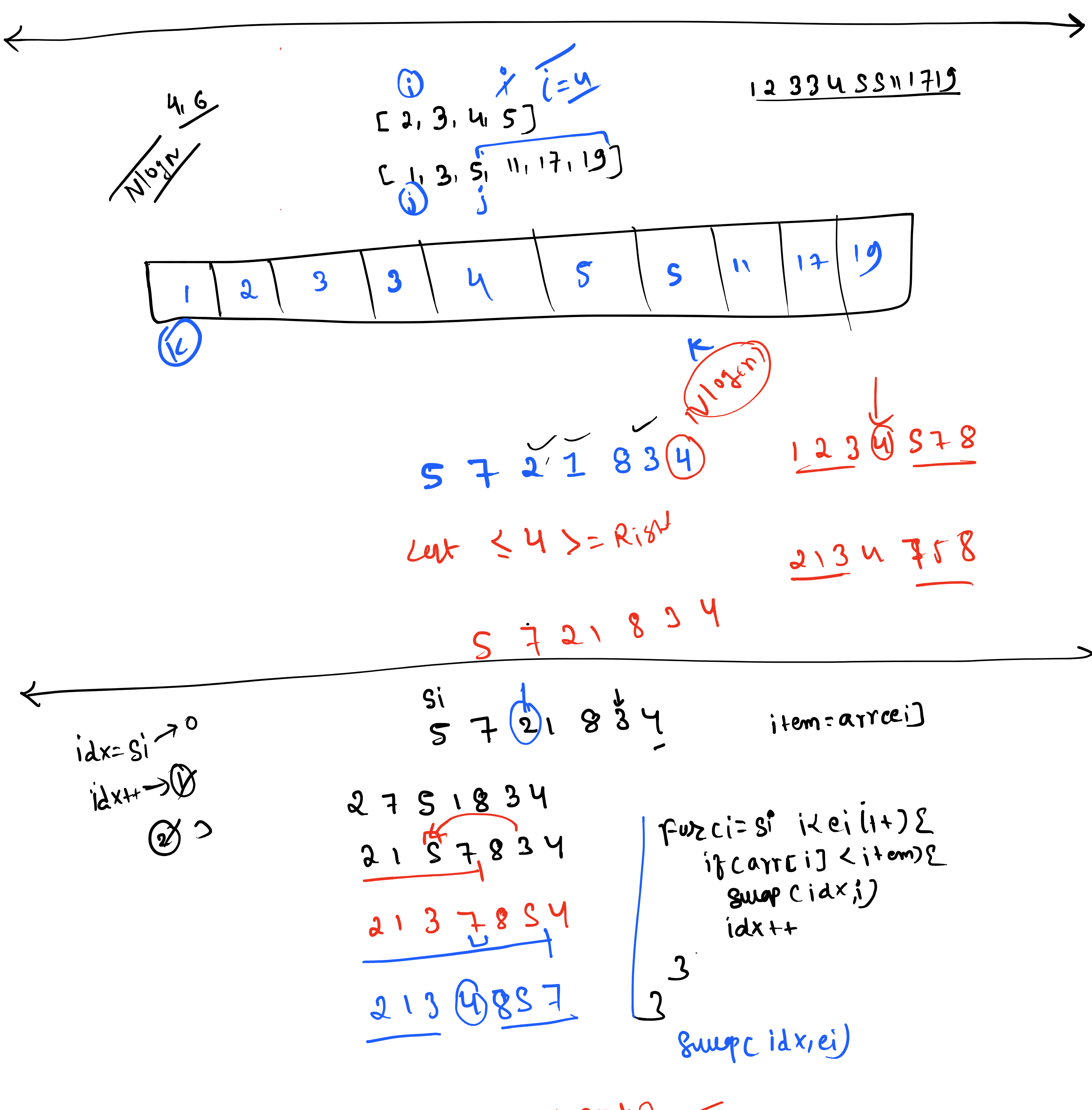
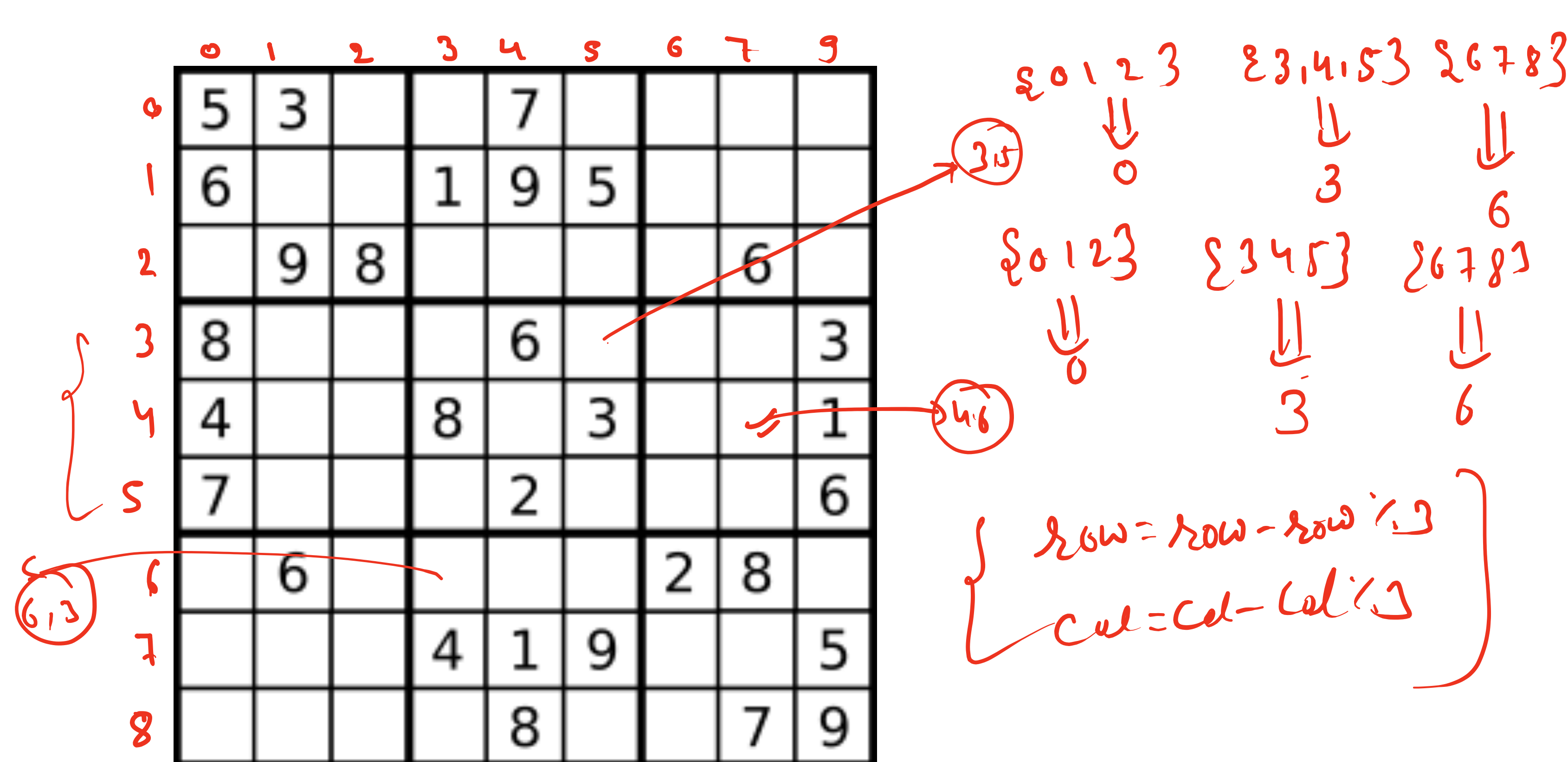
if (col == 0) {
 col = 0;
 row++;
}

0	1	2	3	4	5	6	7	8
0	5	3	1	2	7	6	4	9
1	6	4	7	1	9	5	2	3
2		9	8					6
3	8				6			3
4	4			8		3		1
5	7				2			6
6		6					2	8
7				4	1	9		5
8					8		7	9

row, col ✓

```
public static boolean Print(int[][] grid, int row, int col) {
    if (col == 9) {
        col = 0;
        row++;
    }
    if (row == 9) {
        return true;
    }
    if (grid[row][col] != 0) {
        return Print(grid, row, col + 1);
    }
    else {
        for (int val = 1; val <= 9; val++) {
            if (isitsafe(grid, row, col, val)) {
                grid[row][col] = val;
                boolean ans = Print(grid, row, col + 1);
                if (ans) {
                    return ans;
                }
                grid[row][col] = 0;
            }
        }
        return false;
    }
}
```

0	1	2	3	4	5	6	7	8
0	5	3	1	2	7	6	4	9
1	6			1	9	5		
2		9	8					6
3	8				6			3
4	4			8		3		1
5	7				2			6
6		6					2	8
7				4	1	9		5
8					8		7	9

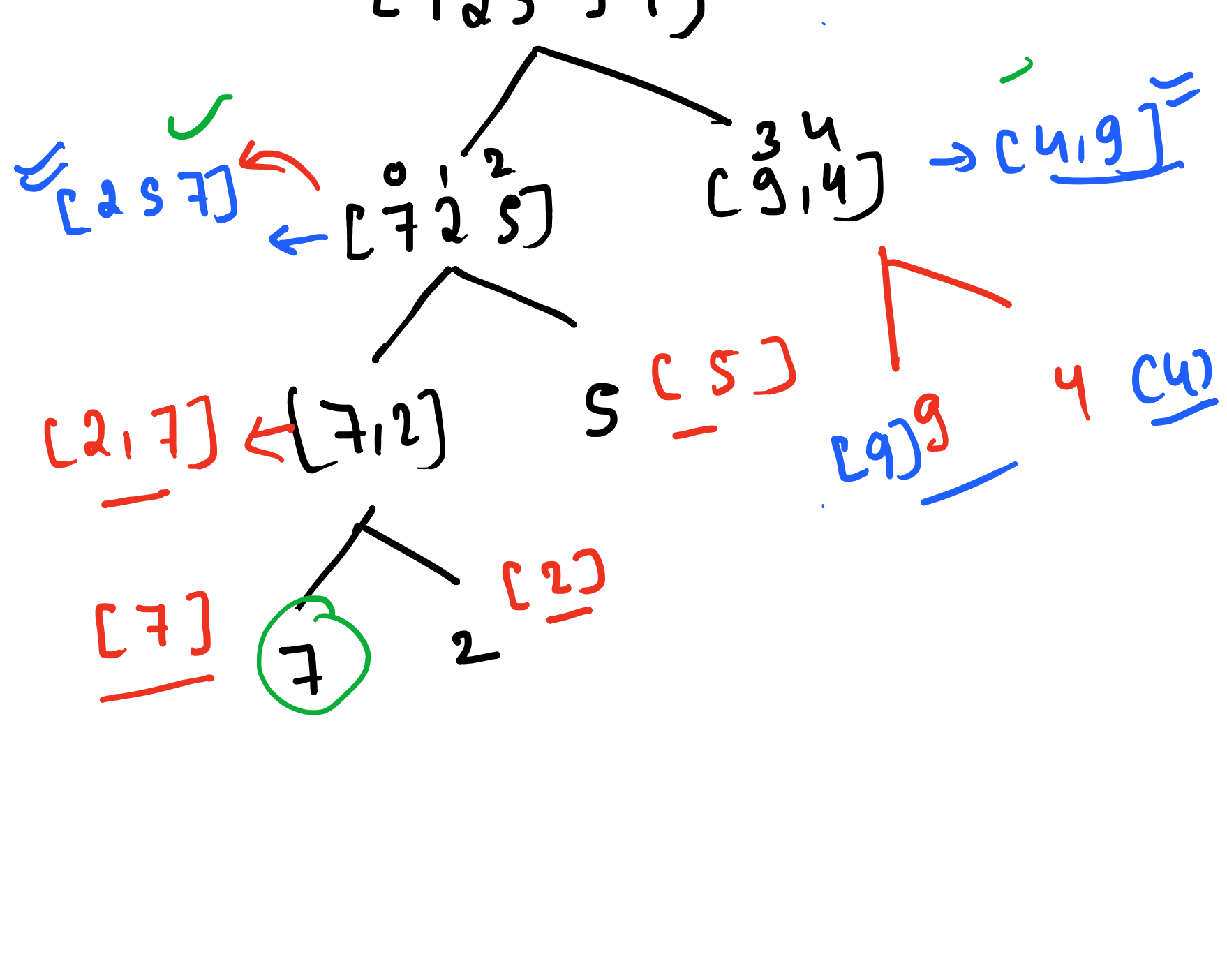
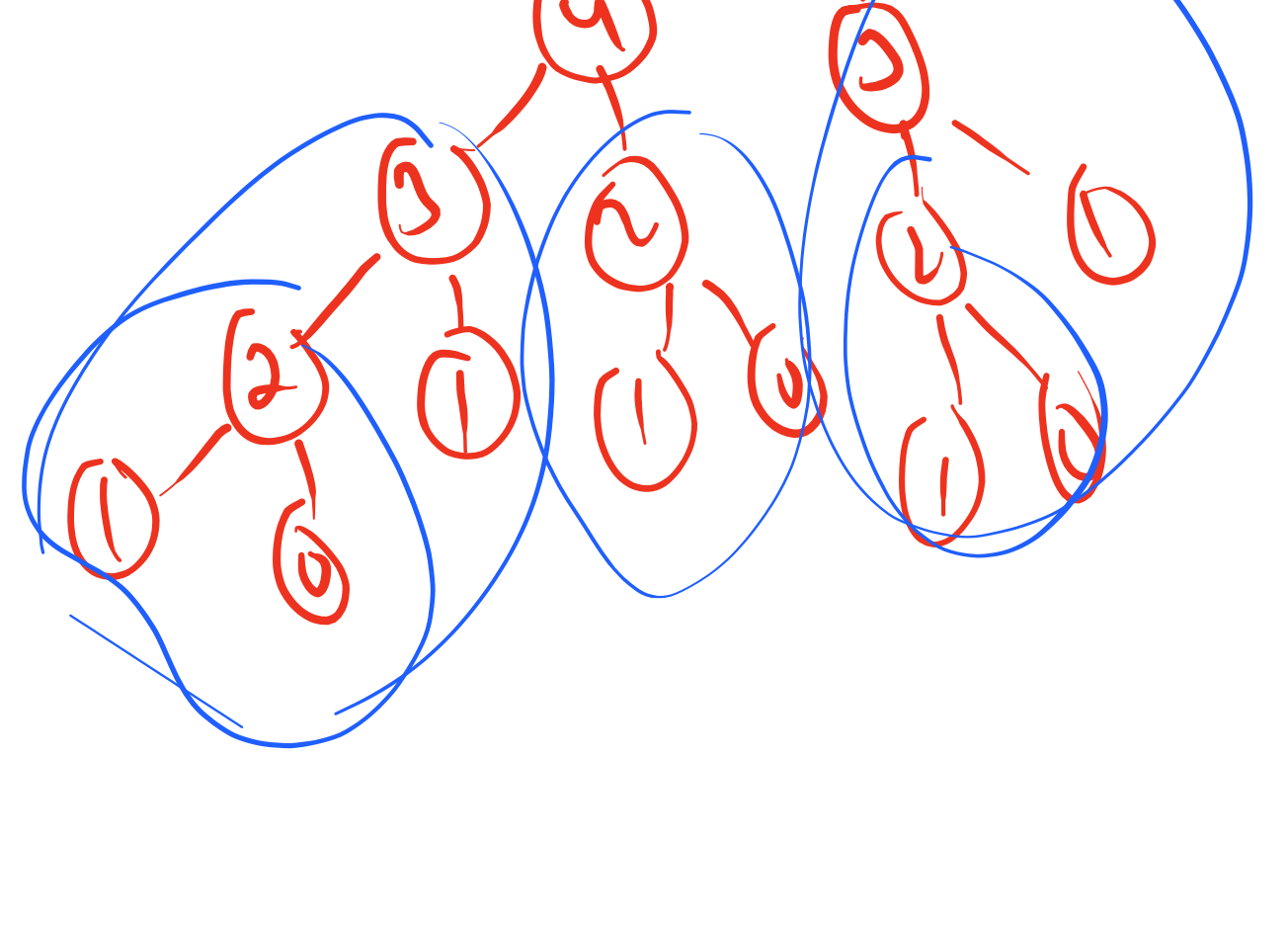


idx = si -> 0
idx++ -> 1
2 >

si 5 7 2 1 8 3 4
2 7 5 1 8 3 4
2 1 5 7 8 3 4
2 1 3 7 8 5 4
2 1 3 4 8 5 7

item = arr[ei]
for (ci = si; ci < ei; ci++) {
 if (arr[ci] < item) {
 swap (ci, ei);
 ei++;
 }
}

Merge -> sort
Quick -> sort



```
public static int[] Sort(int[] arr, int si, int ei) {
    if (si == ei) {
        return arr;
    }
    int mid = (si + ei) / 2;
    int[] f = Sort(arr, si, mid);
    int[] s = Sort(arr, mid + 1, ei);
    return Merge_Two_Array(f, s);
}
```

2 7 5 1 8 3 4
2 1 5 7 8 3 4
2 1 3 7 8 5 4
2 1 3 4 8 5 7

